ICT 3101 OPERATING SYSTEM

Chapter 09

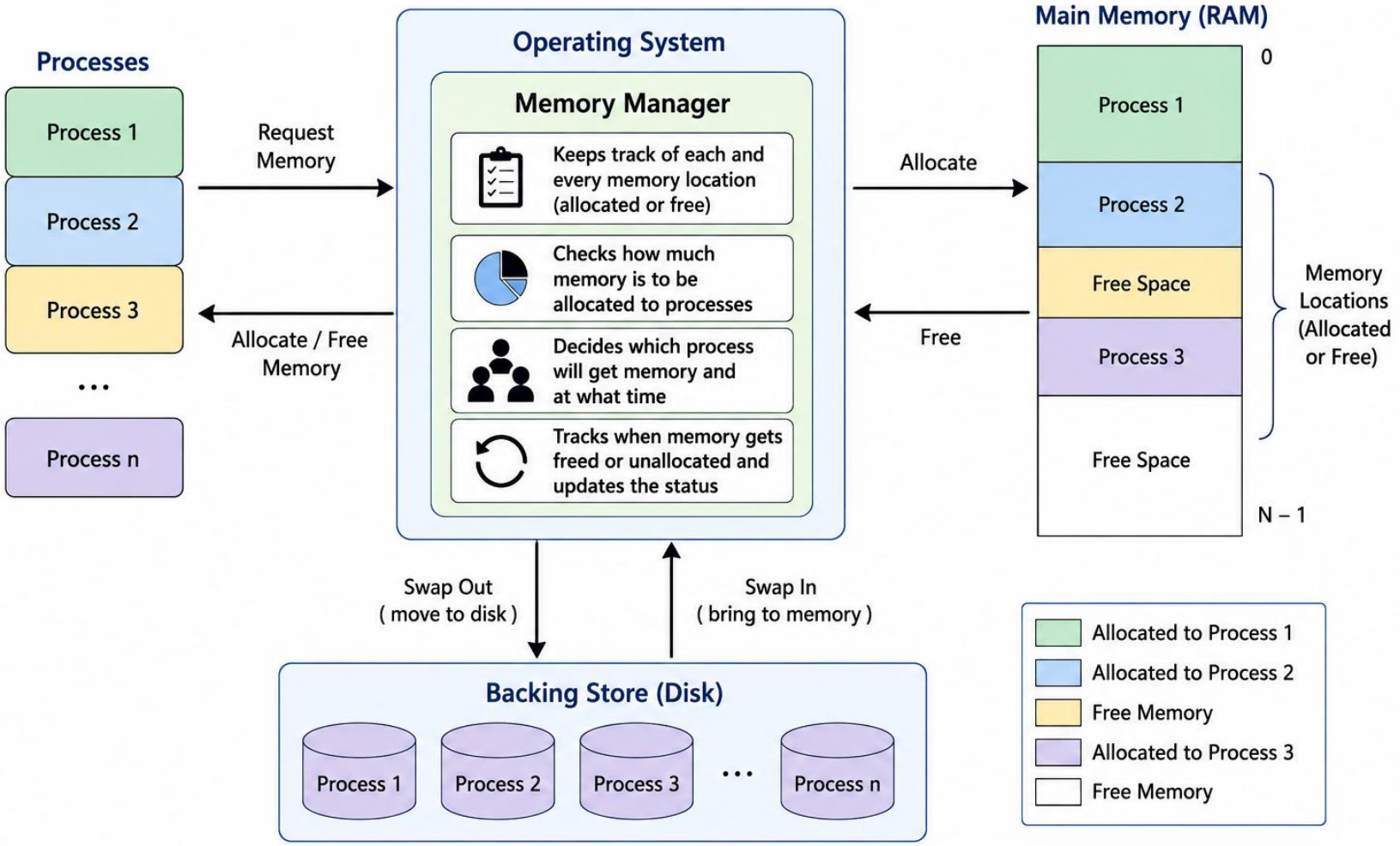
Main Memory

Md Mahmudur Rahman
Assistant Professor
IIT, JU

Memory Management

1. Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution.
2. Memory management keeps track of each and every memory location, regardless of either it is allocated to some process, or it is free.
3. It checks how much memory is to be allocated to processes. It decides which process will get memory at what time.
4. It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

Memory Management in an Operating System



Memory Address

Symbolic addresses

The addresses used in a source code. The variable names, constants, and instruction labels are the basic elements of the symbolic address space.

```
int x = 10;
int y = x + 5;
```

Here:

x , y are symbolic addresses because they are variable names used by the programmer.

Relative addresses

At the time of compilation, a compiler converts symbolic addresses into relative addresses.

Example

Suppose the compiler generates:

Variable	Relative Address
x	100
y	104

Physical addresses

When the program is loaded into RAM, the loader converts relative addresses into physical addresses, which correspond to actual memory locations.

Example

Suppose the program is loaded starting at memory location **5000**.

Variable	Relative Address	Physical Address
x	100	5100
y	104	5104

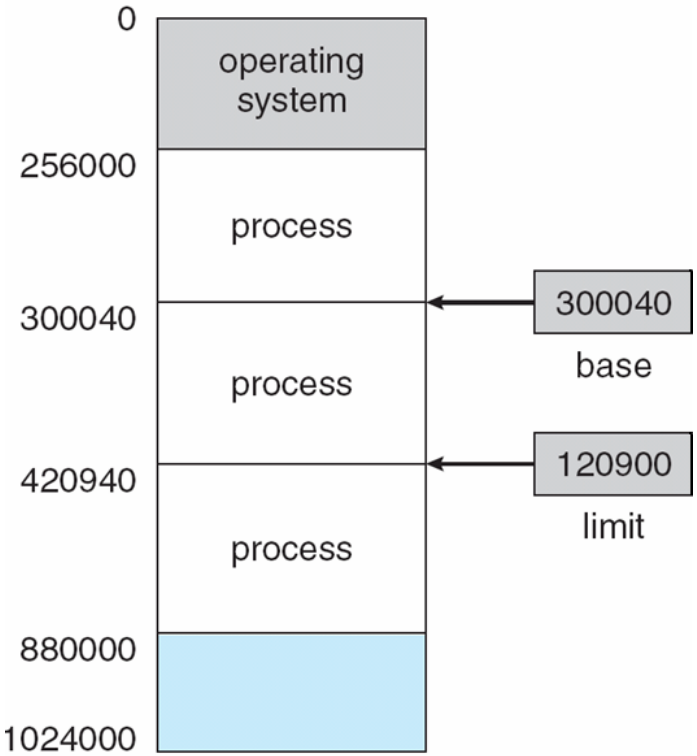
Calculation:

```
Physical Address = Base Address + Relative Address
x = 5000 + 100 = 5100
y = 5000 + 104 = 5104
```

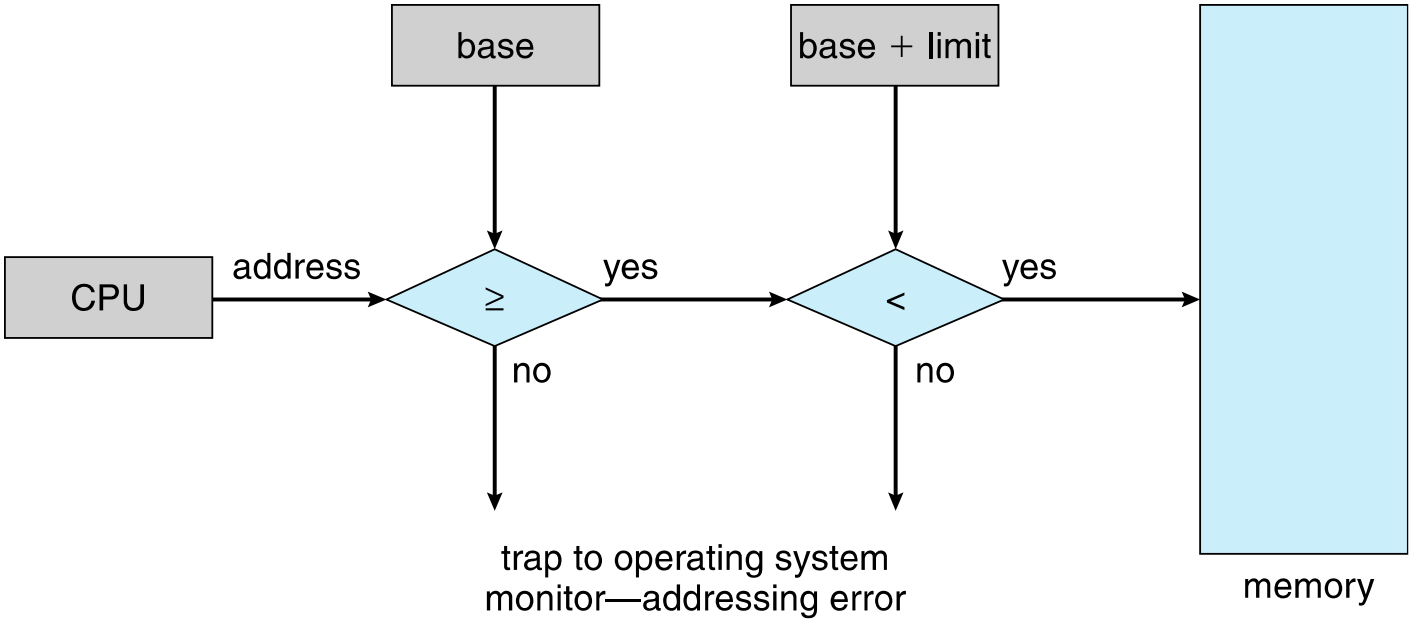
These are the actual locations in RAM accessed by the CPU.

Base and Limit Registers

A pair of **base** and **limit registers** define the logical address space
CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



Hardware Address Protection with Base and Limit Registers



Address Binding

Address binding is the process of mapping the program's logical or virtual addresses to corresponding physical or main memory addresses. In other words, a given logical address is mapped by the MMU (Memory Management Unit) to a physical address.

Programs on disk, ready to be brought into memory to execute form an input queue

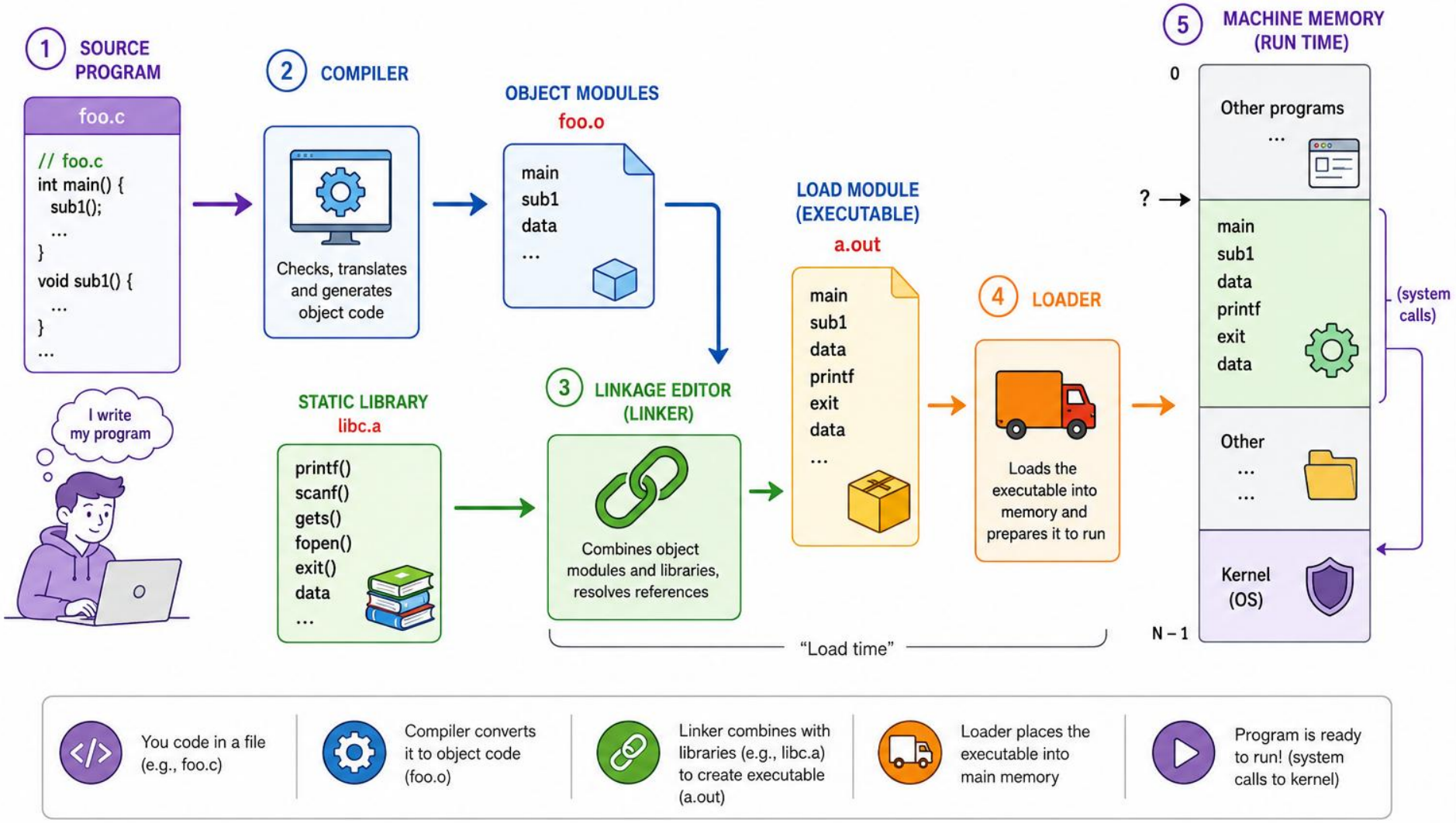
Inconvenient to have first user process physical address always at 0000

Addresses represented in different ways at different stages of a program's life

- Source code addresses are usually symbolic
- A Compiler **bind** this symbolic address to relocatable addresses
  - i.e. "14 bytes from beginning of this module"
- Linker or loader will bind relocatable addresses to absolute addresses
  - i.e. 74014
- Each binding maps one address space to another

From Source to Executable

The Journey of a Program in an Operating System



You code in a file (e.g., foo.c)	Compiler converts it to object code (foo.o)	Linker combines with libraries (e.g., libc.a) to create executable (a.out)	Loader places the executable into main memory	Program is ready to run! (system calls to kernel)

Binding of Instructions and Data to Memory Addresses

Compile time: If memory location known a priori, absolute code can be generated; must recompile code if starting location changes.

- Suppose the program will always start at address:1000
- Compiler generates:

x = 1020
y = 1030

These are absolute addresses.

- **Problem:** If the program must start at: 5000, then you must **recompile**.

Binding of Instructions and Data to Memory Addresses

Load time: If the memory location is not known at compile time, the compiler must generate relocatable code.

- **Loader knows the final location and binds addresses for that location**

Example: Compiler generates:

x = +20
y = +30

These are offsets.

When loading: Base Address = 5000

Loader calculates:

x = 5000 + 20 = 5020
y = 5000 + 30 = 5030

Execution time (Run-Time Binding)**:** If the process can be moved during execution, binding must be delayed until runtime. Need hardware support for address maps (e.g., base and limit registers).

Binding of Instructions and Data to Memory Addresses

Execution time (Run-Time Binding)**:** If the process can be moved during execution, binding must be delayed until runtime. Need hardware support for address maps (e.g., base and limit registers). Modern operating systems use this method.

- **Example**

Initially:

Base = 10000
Logical Address = 50
Physical = 10050

Later OS moves process:

Base = 20000
Logical Address = 50
Physical = 20050

- Program continues without modification.

Memory-Management Unit (MMU)

MMU is the Hardware device that maps **virtual address** to physical address at run time

To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory

- Base register now called **relocation register**
- MS-DOS on Intel 80x86 used 4 relocation registers

The user program deals with logical addresses; it never sees the real physical addresses

- **Execution-time binding occurs when a reference is made to a location in memory**

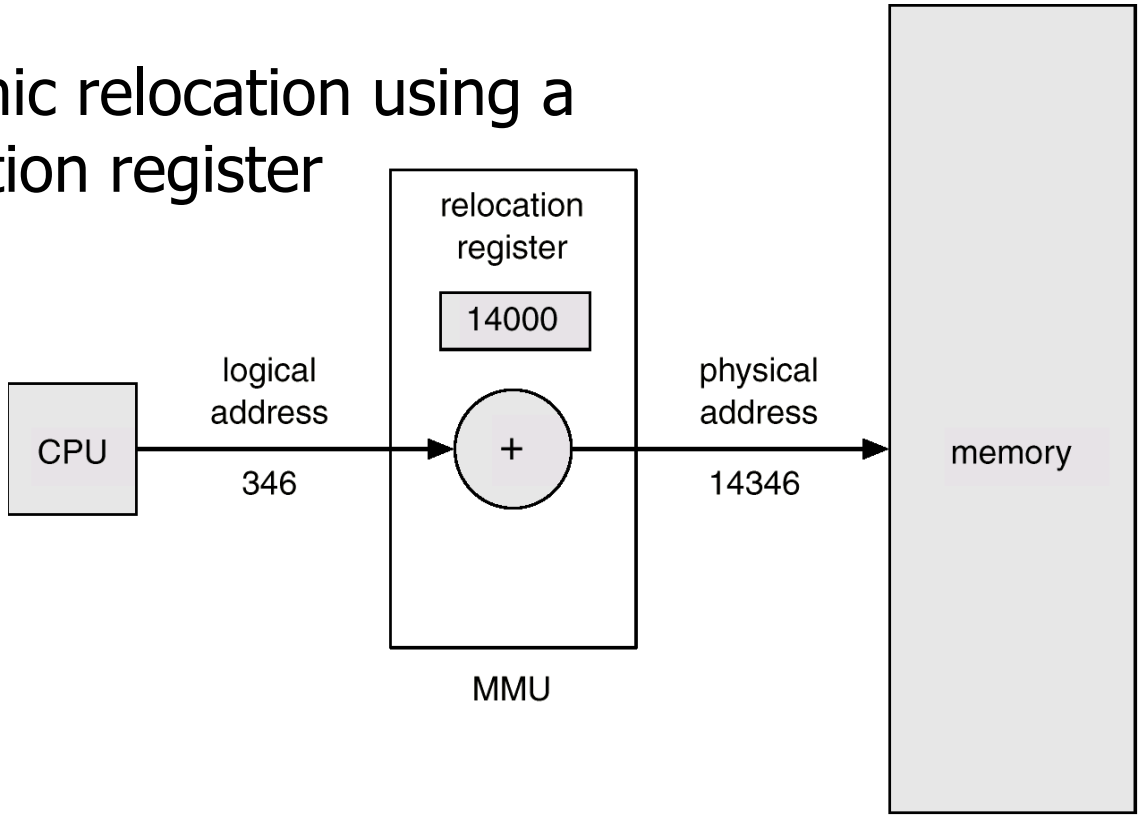
Logical vs. Physical Address Space

Physical address: The actual hardware memory address.

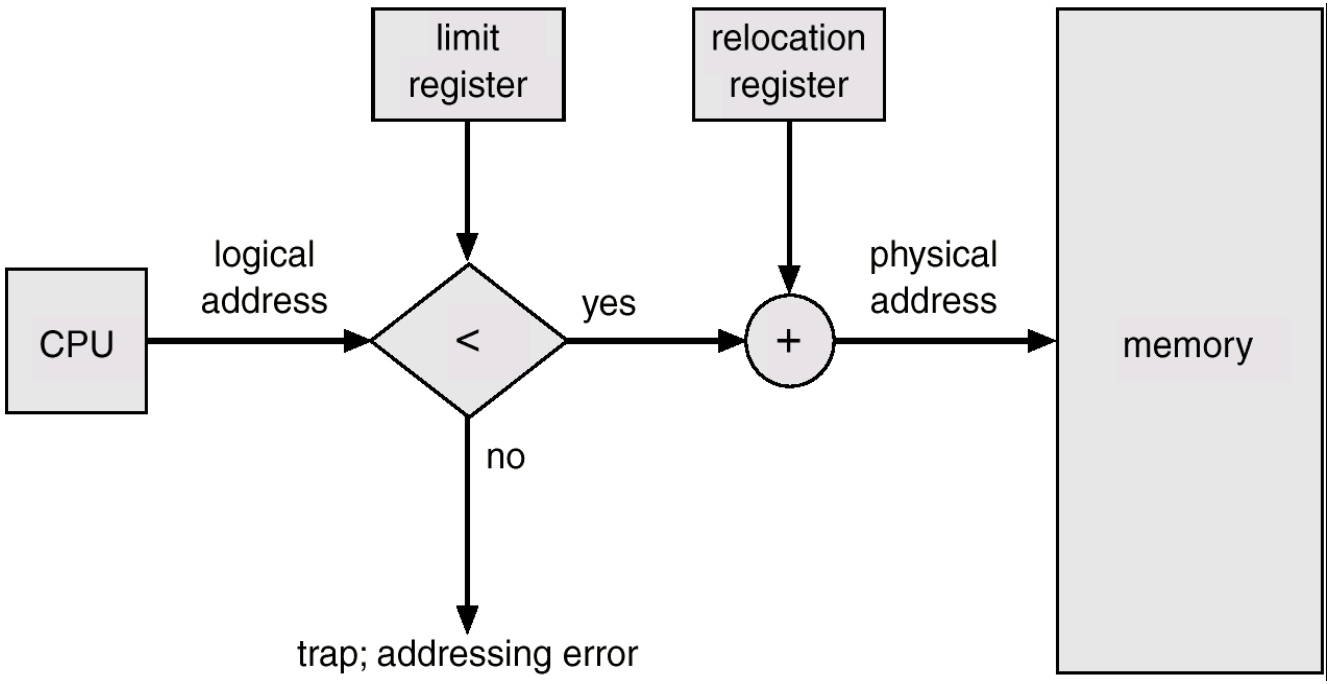
32-bit CPU's physical address 0 ~ 2^{32}-1 (00000000 – FFFFFFFF)

Logical address: Each (relocatable) program assumes the starting location is always 0 and the memory space is much larger than actual memory

Dynamic relocation using a relocation register



Memory Protection



For each process

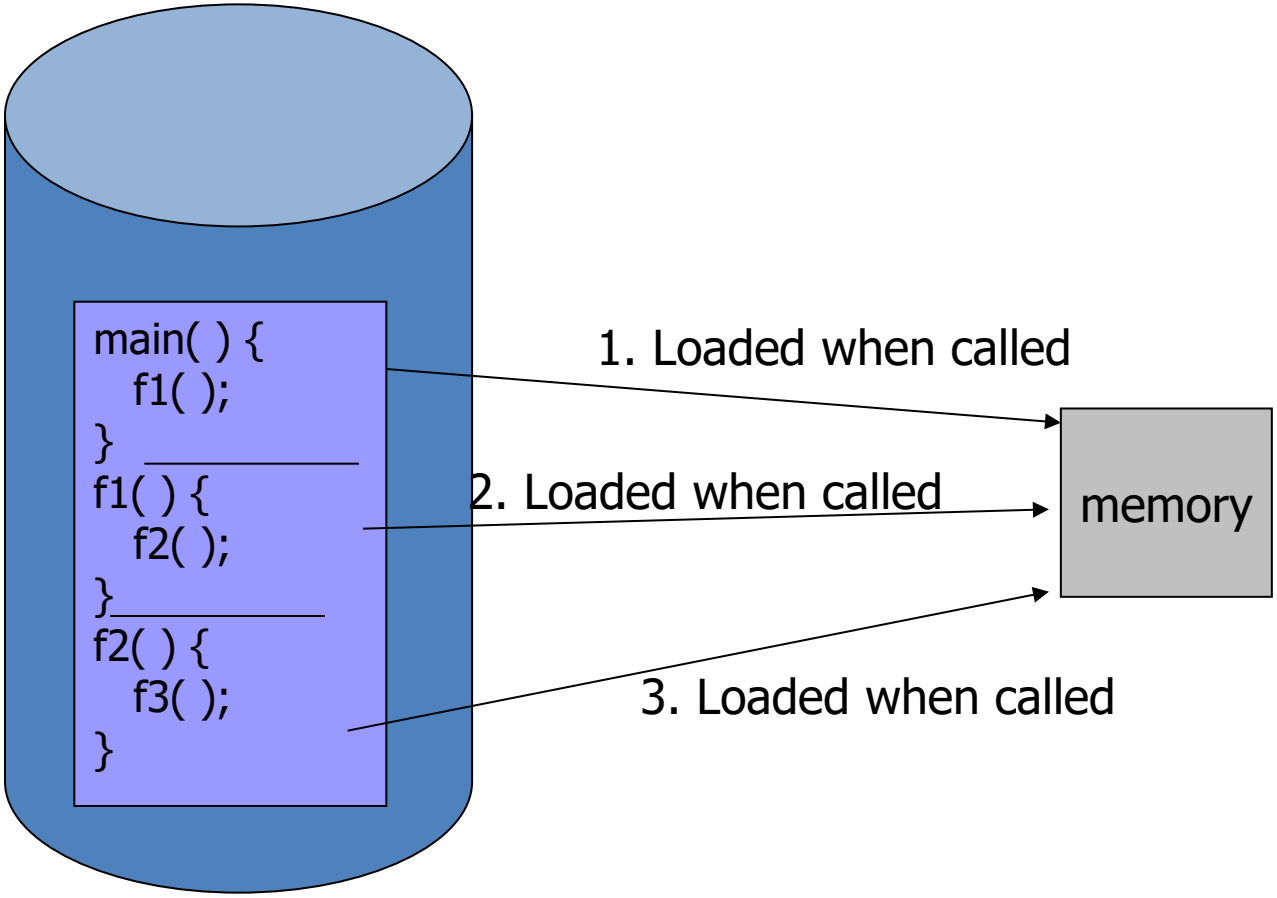
- Logical space is mapped to a contiguous portion of physical space
- A relocation and a limit register are prepared
  - Relocation register = the value of the smallest physical address
  - Limit register = the range of logical address space

Memory Protection

When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values as part of the context switch.
Every address generated by the CPU is checked against these registers.

Dynamic Loading

- Unused routine is never loaded
- Useful when the code size is large



Dynamic Linking

Dynamic Linking is a technique where library functions are linked to a program **during execution (run time)** instead of before execution.

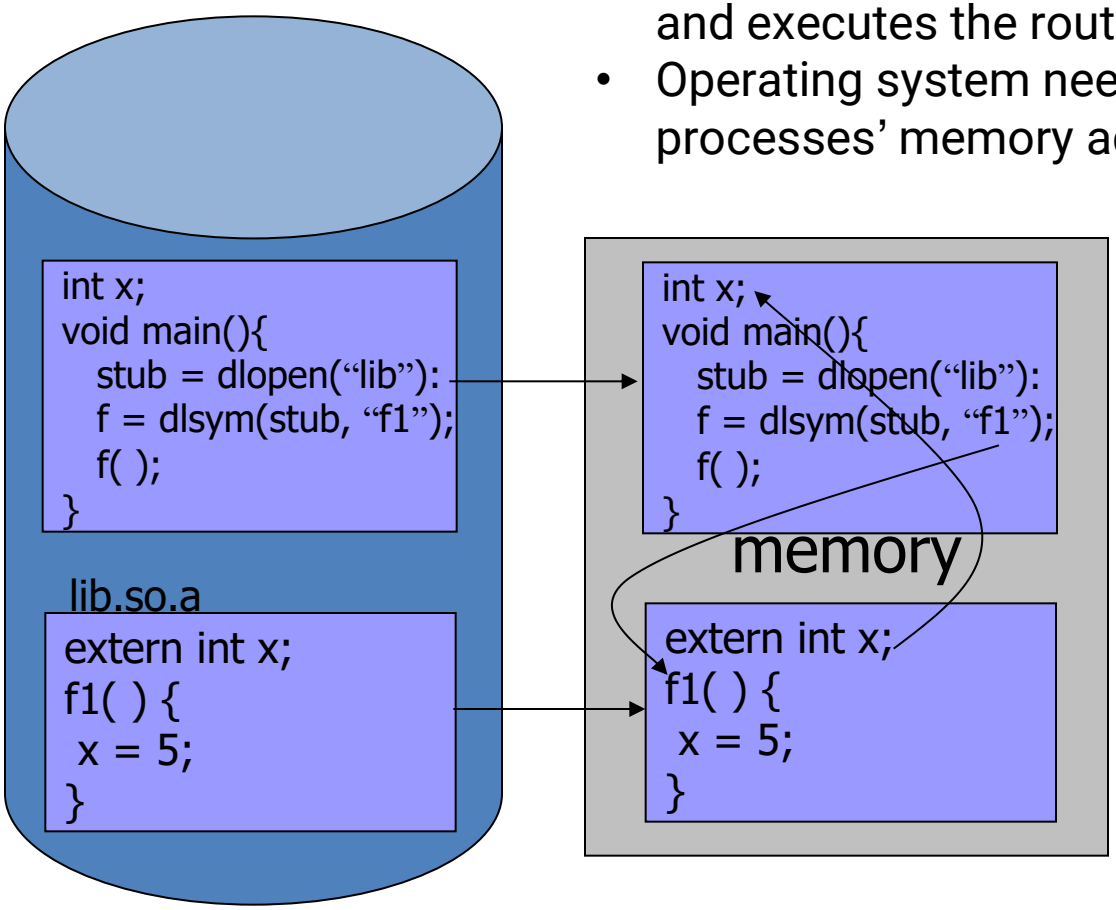
Suppose your program uses the function printf().
Instead of copying the entire printf() code into your executable, the program keeps a small reference called a **stub**.

When the program runs and printf() is needed:

- The **stub** checks where the library function is located.
- The operating system loads the required library (if not already loaded).
- The stub obtains the address of the function.
- The function is executed.
- Future calls use the same address directly.

Dynamic Linking

- Linking postponed until execution time.
- Small piece of code, *stub*, used to locate the appropriate memory-resident library routine.
- Stub replaces itself with the address of the routine, and executes the routine.
- Operating system needs to check if routine is in processes' memory address



Example

Suppose: `printf`("Hello");

Without Dynamic Linking (Static Linking)

Executable
- Main Program
- `printf()` code

The executable becomes larger because it contains a copy of `printf()`.

With Dynamic Linking

Executable
- Main Program
- Stub for `printf()`

The actual `printf()` remains in a shared library file such as:

`libc.so`

and is loaded only when needed.

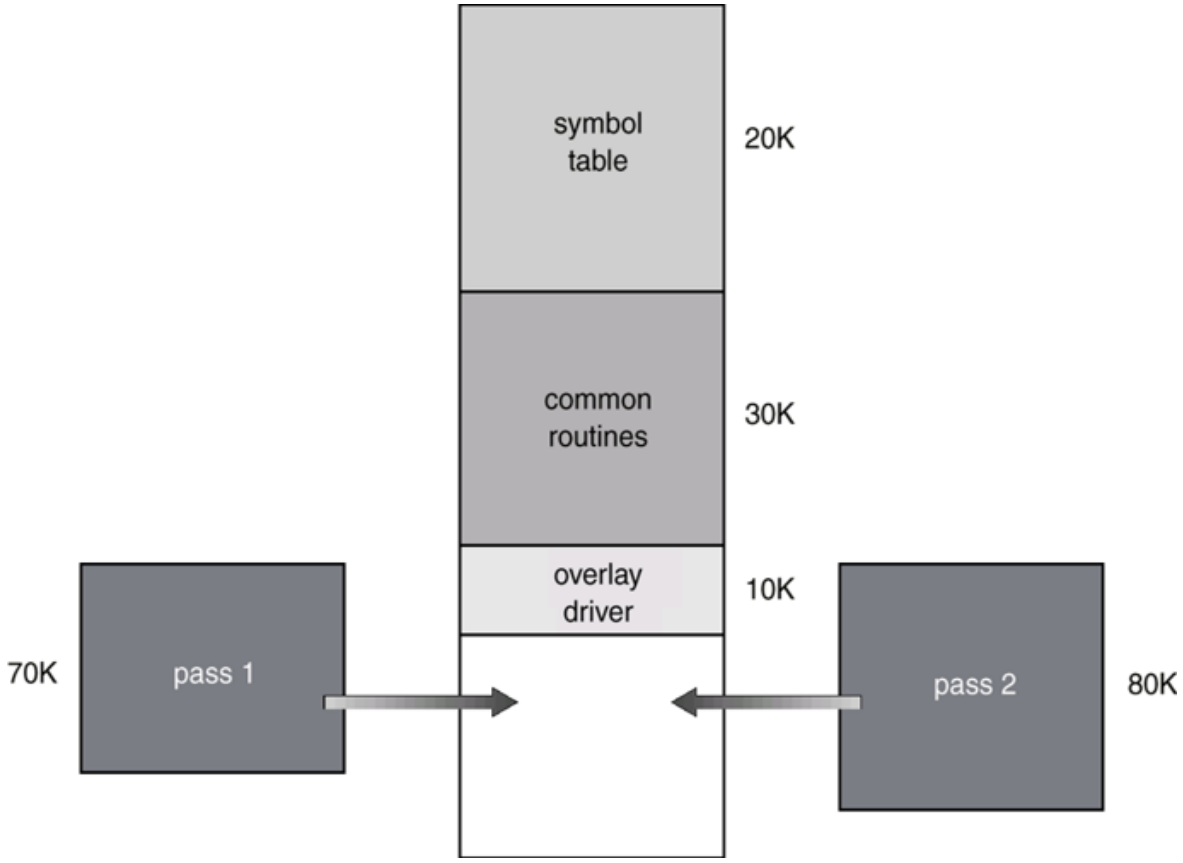
Overlays

1. Overlay is a memory-management technique used when a program is larger than the available memory.
2. Instead of loading the entire program into memory at once, only the currently needed part of the program is loaded. When another part is needed, it replaces the previous part.

Special relocation and linking algorithms are needed to construct the overlays

Why Use Overlays?

Suppose:
Available memory = **60 KB**
Program size = **130 KB**
Since the whole program cannot fit into memory, we divide it into smaller modules called **overlays**.



The figure shows:

```
Symbol Table       20 KB
Common Routines    30 KB
Overlay Driver     10 KB
-----------------------
Total Fixed Part   60 KB
```

These components must always remain in memory.

Now there are two large modules:

```
Pass 1 = 70 KB
Pass 2 = 80 KB
```

Only one of them is needed at a time.

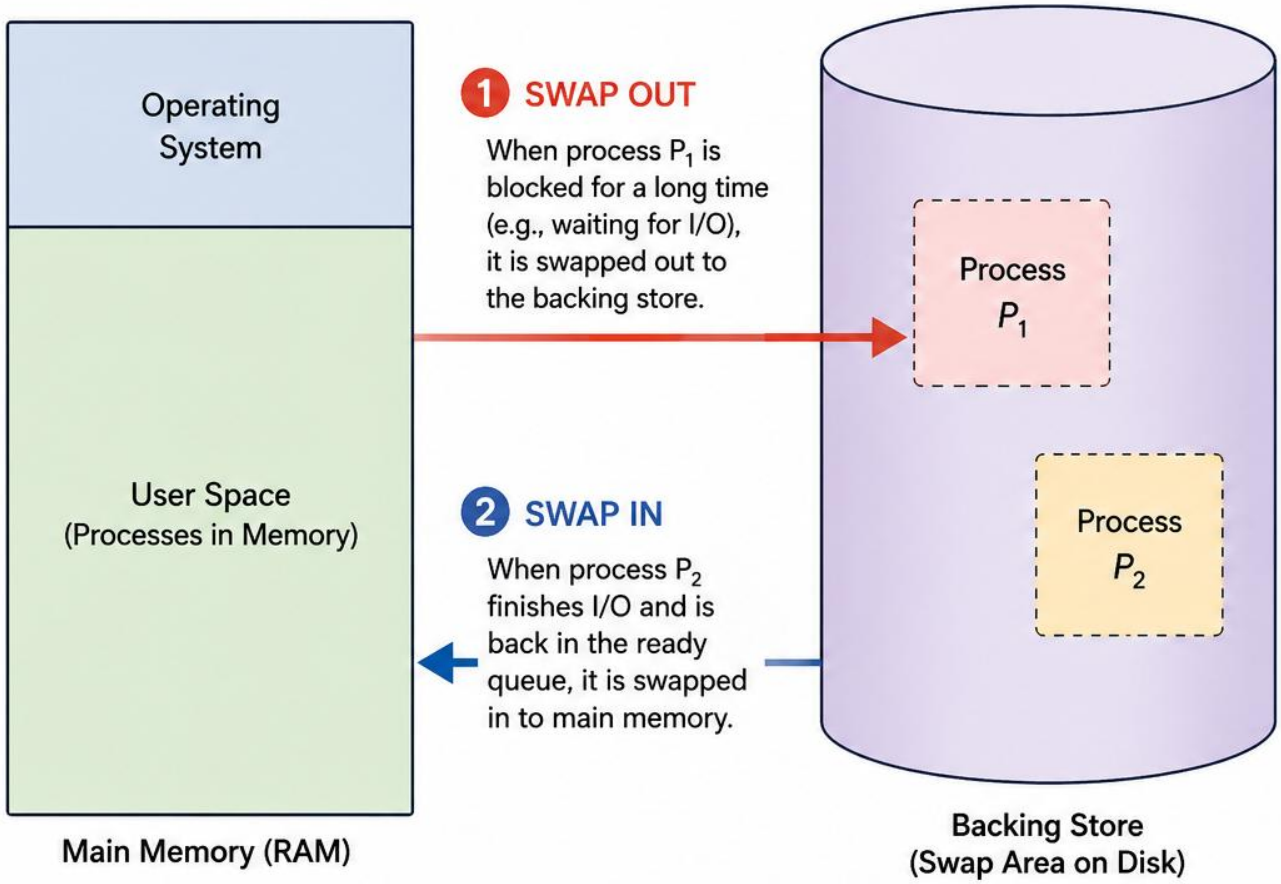
Therefore: Memory

```
+------------------+
| Symbol Table     | 20K
+------------------+
| Common Routines  | 30K
+------------------+
| Overlay Driver   | 10K
+------------------+
| Pass 1 OR Pass 2 |
+------------------+
```

When **Pass 1** finishes, it is removed and **Pass 2** is loaded into the same memory area.

Swapping

Swapping: Moving Processes Between Memory and Disk



Explanation

- Swapping is a memory management technique where entire processes are moved between main **memory (RAM) and** disk (backing store).
- **1 Swap Out:** If a process (P_1) is blocked for a long time (e.g., for I/O), it is removed from RAM and stored in the backing store.
- **2 Swap In:** When another process (P_2) is ready to run, it is brought from the backing store back into RAM.
- In Unix, use the top command to view which processes are currently swapped out.

Key Idea: Swapping frees up RAM by temporarily moving inactive (blocked) processes to disk and bringing back active (ready) processes when needed.

Memory Allocation (Fixed sized partition)

- The program always executes in main memory.
- The size of main memory affects degree of Multi programming to most of the extant.
- If the size of the main memory is larger than CPU can load more processes in the main memory at the same time and therefore will increase degree of Multi programming as well as CPU utilization.

Memory Allocation (Fixed sized partition)

Case 1: Memory = 4 MB

Suppose:

- Process size = **4 MB**
- Main memory = **4 MB**
- Only **one process** can stay in memory.

Let:

- **P** = Probability that a process is waiting for I/O.
- Assume **P = 70% = 0.7**

This means:

70% of the time → Process waits for I/O

30% of the time → CPU is working

Therefore:

CPU Utilization = 1 - P

= 1 - 0.7 = 0.3 = **30%**

Case 2: Memory = 8 MB

Now memory is doubled.

- Process size = 4 MB
- Memory size = 8 MB

Two processes can fit in memory simultaneously.

The CPU becomes idle only if **both processes are waiting for I/O at the same time**.

Probability:

P(both waiting)

= P × P = P^2

For P = 0.7:

P^2 = 0.7 × 0.7 = 0.49

Therefore:

CPU Utilization

= 1 - P^2 = 1 - 0.49

= 0.51 = **51%**

CPU utilization increases from **30% to 51%**.

CPU Utilization = 1 - P^n

Memory Allocation (Fixed Sized Partition)

More Memory → More Processes in Memory → Less CPU Idle Time → Higher CPU Utilization

CASE 1: MEMORY = 4 MB

Process size = 4 MB

Main memory = 4 MB

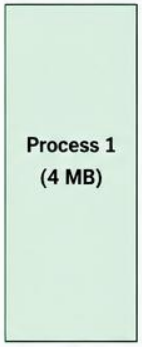
Only one process can be in memory at a time.

Let

P = Probability that a process is waiting for I/O

Assume P = 70%

Memory (4 MB)



Timeline (One Process)

CPU is idle when the only process is waiting for I/O.

CPU Utilization = 1 − P

= 1 − 0.7

= 0.3 = 30%

CASE 2: MEMORY = 8 MB

Process size = 4 MB

Main memory = 8 MB

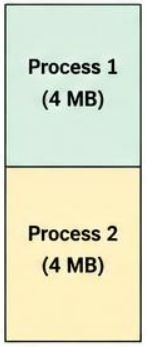
Two processes can be in memory at the same time.

Let

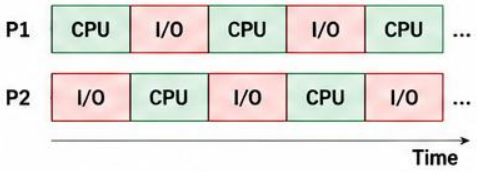
P = Probability that a process is waiting for I/O

Assume P = 70%

Memory (8 MB)



Timeline (Two Processes)



CPU is idle only when BOTH processes are waiting for I/O at the same time.

CPU Utilization = $1 - P^2$

= $1 - (0.7)^2$

= 1 − 0.49 = 0.51 = 51%

GENERAL FORMULA

If n processes can reside in memory at the same time,

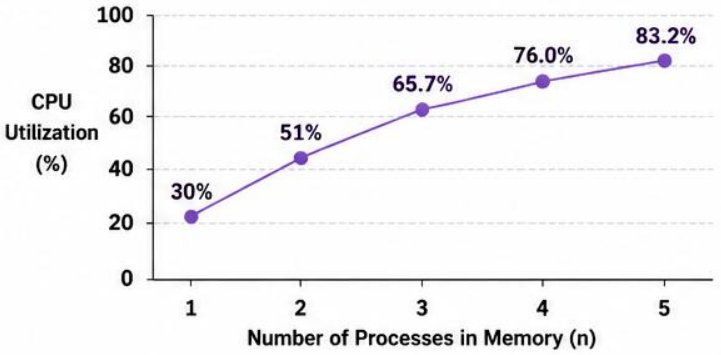
CPU Utilization = $1 - P^n$

(CPU is idle only when all n processes are waiting for I/O)

MORE EXAMPLES (P = 70% = 0.7)

Memory Size (MB)	Processes in Memory (n)	Probability CPU Idle (P^n)	CPU Utilization ($1 - P^n$)
4	1	$0.7^1 = 0.70$	0.30 = 30%
8	2	$0.7^2 = 0.49$	0.51 = 51%
12	3	$0.7^3 = 0.343$	0.657 = 65.7%
16	4	$0.7^4 = 0.2401$	0.7599 = 76.0%
20	5	$0.7^5 = 0.1681$	0.8319 = 83.2%

CPU UTILIZATION vs NUMBER OF PROCESSES IN MEMORY (P = 70%)



As memory increases, more processes can reside in memory, reducing CPU idle time and increasing CPU utilization.

Key Takeaway: With fixed sized partitions, increasing memory size allows more processes to stay in memory simultaneously. CPU becomes idle only when all of them are waiting for I/O. Therefore, CPU utilization increases.

Memory Allocation (Fixed sized partition)

- The earliest and one of the simplest technique which can be used to load more than one processes into the main memory is Fixed partitioning or Contiguous memory allocation.
- In this technique, the main memory is divided into partitions of equal or different sizes.
- The operating system always resides in the first partition while the other partitions can be used to store user processes. The memory is assigned to the processes in contiguous way.
- In fixed partitioning, the partitions cannot overlap.
- A process must be contiguously present in a partition for the execution.

Memory Allocation (Fixed sized partition)

There are various cons of using this technique.

Internal Fragmentation

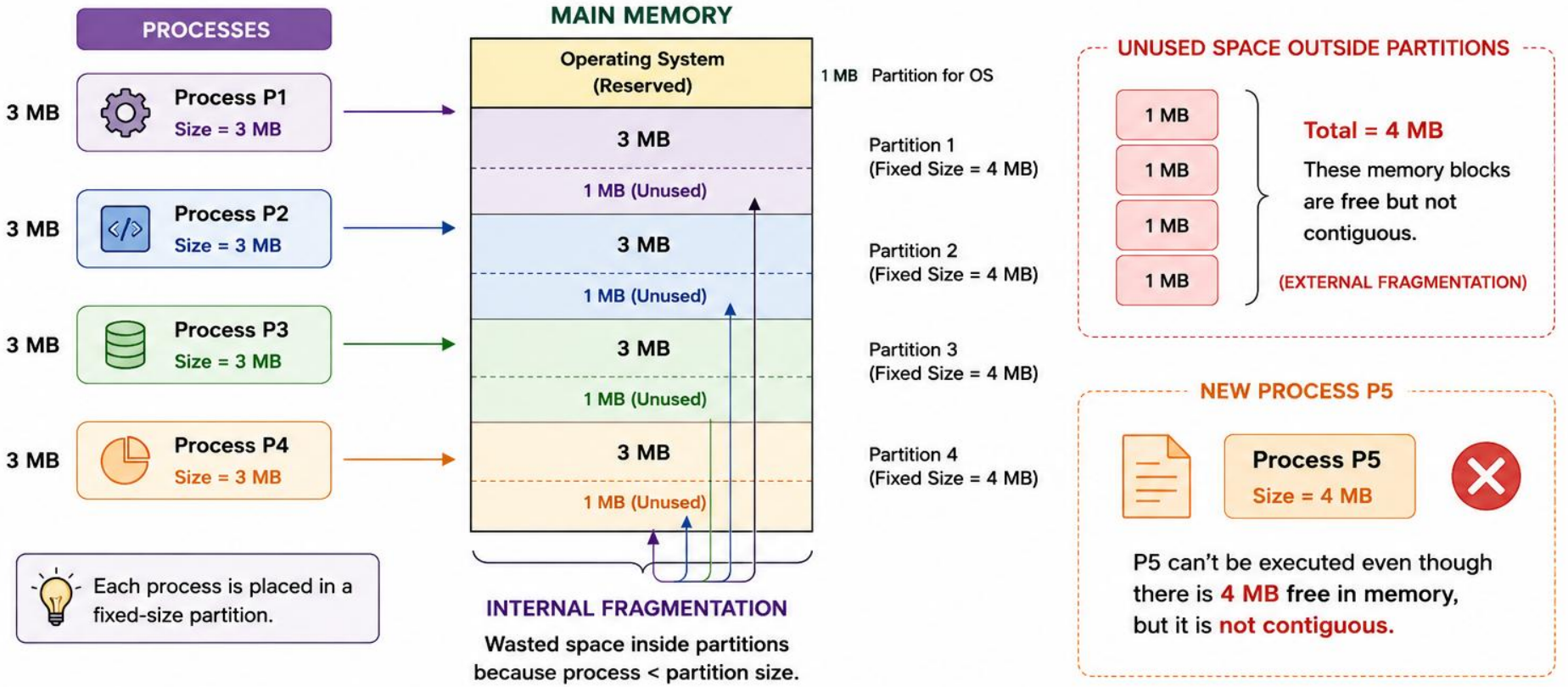
1. If the size of the process is lesser then the total size of the partition then some size of the partition get wasted and remain unused. This is wastage of the memory and called internal fragmentation.
2. As shown in the image below, the 4 MB partition is used to load only 3 MB process and the remaining 1 MB got wasted.

External Fragmentation

1. The total unused space of various partitions cannot be used to load the processes even though there is space available, but not in a contiguous form.
2. As shown in the image below, the remaining 1 MB space of each partition cannot be used as a unit to store a 4 MB process. Although sufficient space is available to load the process, it will not be loaded.

FIXED PARTITIONING

Contiguous Memory Allocation



KEY TAKEAWAYS

- Memory is divided into fixed-size partitions.
- Processes are loaded into partitions that are large enough to hold them.
- Internal fragmentation occurs when partition size > process size.
- External fragmentation occurs when free memory is not contiguous.

Memory Allocation (Fixed sized partition)

Limitation on the size of the process

- If the process size exceeds the maximum partition size, the process cannot be loaded into memory.
- Therefore, a limitation can be imposed on the process size, which is it cannot be larger than the size of the largest partition.

Degree of multiprogramming is less

- By Degree of multi programming, we simply mean the maximum number of processes that can be loaded into the memory at the same time.
- In fixed partitioning, the degree of multiprogramming is fixed and very less due to the fact that the size of the partition cannot be varied according to the size of processes.

Variable-Sized (Dynamic Partitioning)

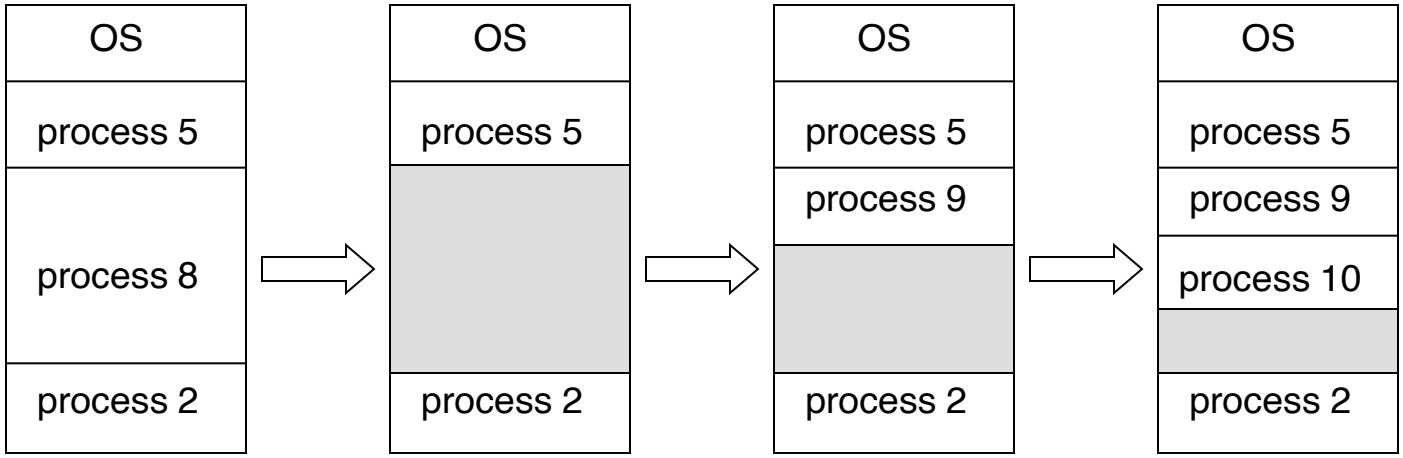
- The OS keeps a table indicating which parts of memory are available, and which are occupied
- Initially all memory is available for the user processes and is considered as one large block of available memory, **“HOLE”.**
- When a process arrives and needs memory, we search for a hole large enough for this process.
- If we find one, we allocate only as much memory as is needed.

Variable-Sized (Dynamic Partitioning)

Whenever one of running processes, (p8) is terminated

- Find a ready process whose size is best fit to the hole, (p9)
- Allocate it from the top of hole

If there is still an available hole, repeat the above (for p10).
Any size of processes, (up to the physical memory size) can be allocated.



Dynamic Partitioning- Advantages

No Internal Fragmentation

Given the fact that the partitions in dynamic partitioning are created according to the need of the process, It is clear that there will not be any internal fragmentation because there will not be any unused remaining space in the partition.

No Limitation on the size of the process

In Fixed partitioning, the process with the size greater than the size of the largest partition could not be executed due to the lack of sufficient contiguous memory. Here, In Dynamic partitioning, the process size can't be restricted since the partition size is decided according to the process size.

Degree of multiprogramming is dynamic

Due to the absence of internal fragmentation, there will not be any unused space in the partition hence more processes can be loaded in the memory at the same time.

Dynamic Partitioning- Disadvantages

External Fragmentation

- Absence of internal fragmentation doesn't mean that there will not be external fragmentation.
- Let's consider three processes P1 (1 MB) and P2 (3 MB) and P3 (1 MB) are being loaded in the respective partitions of the main memory.
- After some time, P1 and P3 got completed, and their assigned spaces were freed. Now there are two unused partitions (1 MB and 1 MB) available in the main memory but they cannot be used to load a 2 MB process in the memory since they are not contiguously located.
- The rule says that the process must be **contiguously** present in the main memory to get executed.

Dynamic Partitioning- Disadvantages

Complex Memory Allocation

- In Fixed partitioning, the list of partitions is made once and will never change, but in dynamic partitioning, the allocation and deallocation is very complex since the partition size will be varied every time it is assigned to a new process. OS has to keep track of all the partitions.
- Since the allocation and deallocation are done very frequently in dynamic memory allocation, and the partition size will be changed at each time, it is going to be very difficult for the OS to manage everything.

Compaction

- We can also use compaction to minimize the probability of external fragmentation. In compaction, all the free partitions are made contiguous and all the loaded partitions are brought together.

- By applying this technique, we can store larger processes in memory. The free partitions are merged, allowing them to be allocated according to the needs of new processes. This technique is also called defragmentation

MEMORY COMPACTION (DEFRAGMENTATION)

Making Free Memory Contiguous

Compaction is used to reduce **EXTERNAL FRAGMENTATION**.
All the free (unallocated) partitions are merged to form one contiguous block, and all the allocated partitions are moved together.

This creates a large continuous free space that can be used to allocate memory to new processes.
This technique is also called **DEFRAGMENTATION**.

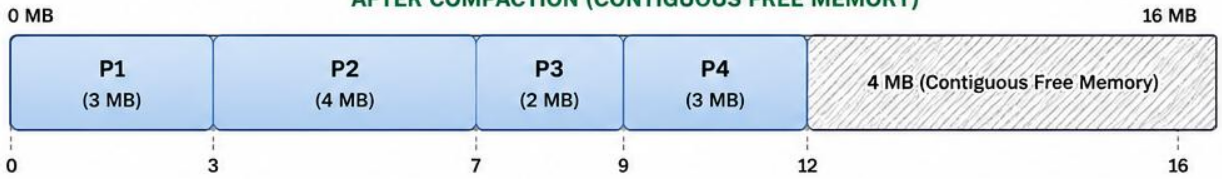
Allocated (Processes) | Free Memory | Movement during compaction | MB = Megabyte

BEFORE COMPACTION (FRAGMENTED MEMORY)

PROBLEM: EXTERNAL FRAGMENTATION

Total free memory = 2 + 1 + 3 + 1 = 7 MB
But it is in multiple small holes.
A new process of size 6 MB cannot be allocated.

AFTER COMPACTION (CONTIGUOUS FREE MEMORY)



BENEFIT AFTER COMPACTION

- All free memory is now one big block (4 MB).
- A new process of size up to 4 MB can now be allocated.
- External fragmentation is eliminated.

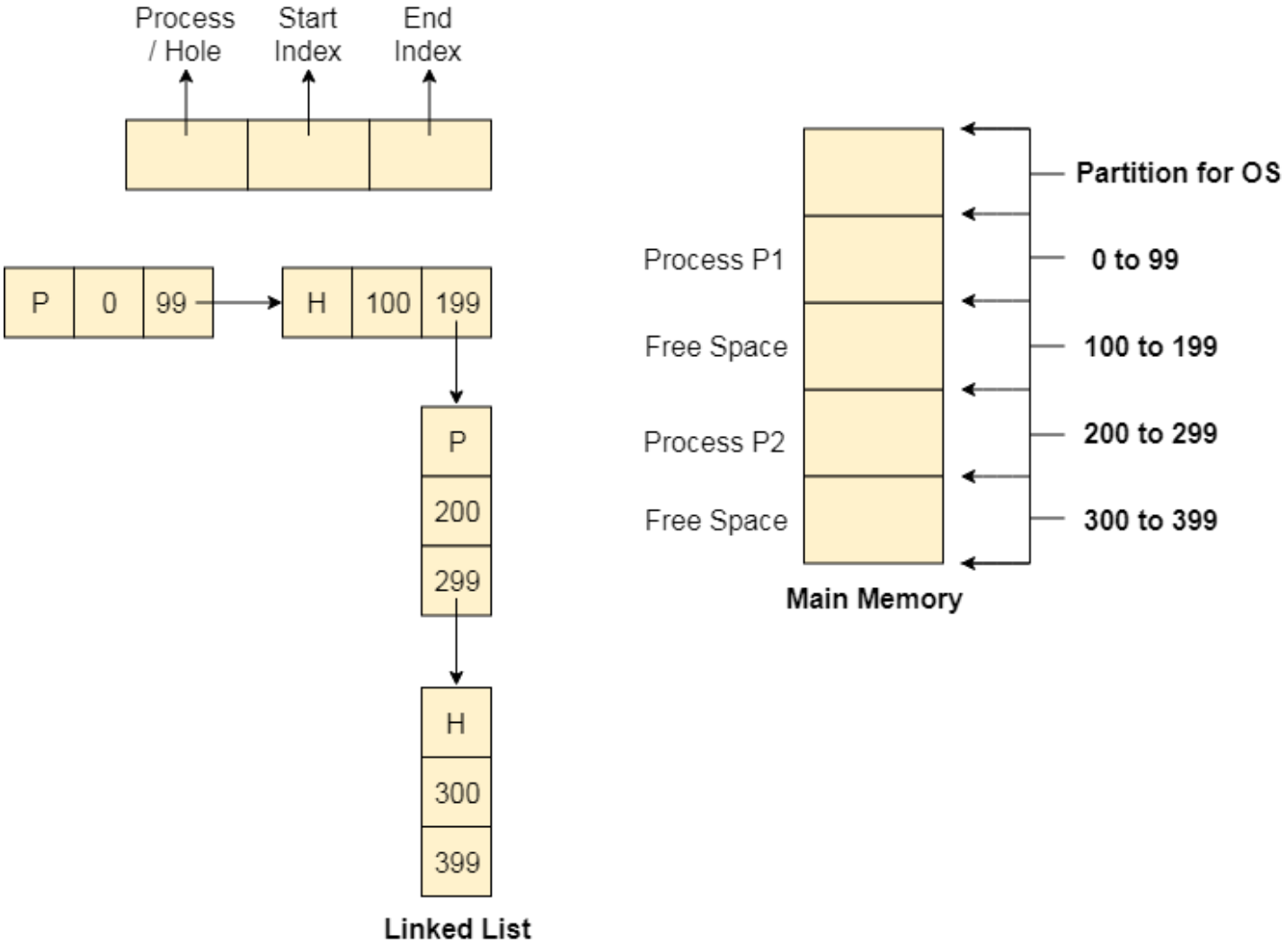
HOW COMPACTION WORKS

1. **Identify free holes**
   OS scans memory and finds all free (unallocated) partitions.
2. **Move allocated blocks**
   All processes are shifted towards one end of memory (usually the beginning).
3. **Merge free spaces**
   All the small free holes are combined into one large contiguous block.
4. **Allocate efficiently**
   Now larger processes can be loaded into the contiguous free space.

Dynamic Storage-Allocation Problem

- The Main concern for dynamic partitioning is keeping track of all the free and allocated partitions. However, the Operating system uses Linked List for this task.
- In this approach, the Operating system maintains a linked list where each node represents each partition. Every node has three fields.
- First field of the node stores a flag bit which shows whether the partition is a hole or some process is inside.
- Second field stores the starting index of the partition.
- Third filed stores the end index of the partition.

Linked List for Dynamic Partitioning



Partitioning Algorithms

There are various algorithms which are implemented by the Operating System in order to find out the holes in the linked list and allocate them to the processes.

First-fit: Allocate the first hole that is big enough. (Fastest search)

Best-fit: Allocate the smallest hole that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole. (Best memory usage)

Worst-fit: Allocate the largest hole; must also search entire list. Produces the largest leftover hole (that could be used effectively later)

Example

List of Jobs	Size	Turnaround
Job 1	100k	3
Job 2	10k	1
Job 3	35k	2
Job 4	15k	1
Job 5	23k	2
Job 6	6k	1
Job 7	25k	1
Job 8	55k	2
Job 9	88k	3
Job 10	100k	3

Memory Block	Size
Block 1	50k
Block 2	200k
Block 3	70k
Block 4	115k
Block 5	15k

Example (First Fit)

Memory Block	Size	List of Jobs
Block 1	50k	**Job 2**
Block 2	200k	**Job 1**
Block 3	70k	**Job 3**
Block 4	115k	**Job 4**
Block 5	15k	**Job 6**

Memory Block	Size	List of Jobs
Block 1	50k	**Job 5**
Block 2	200k	**Job 1**
Block 3	70k	**Job 3**
Block 4	115k	**Job 7**
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	**Job 5**
Block 2	200k	**Job 1**
Block 3	70k	**Job 8**
Block 4	115k	**Job 9**
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	**Job 10**
Block 3	70k	**Job 8**
Block 4	115k	**Job 9**
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	**Job 10**
Block 3	70k	
Block 4	115k	**Job 9**
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	**Job 10**
Block 3	70k	
Block 4	115k	
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	
Block 3	70k	
Block 4	115k	
Block 5	15k	

Example (Best Fit)

Memory Block	Size	List of Jobs
Block 1	50k	**Job 3**
Block 2	200k	**Job 5**
Block 3	70k	**Job 4**
Block 4	115k	**Job 1**
Block 5	15k	**Job 2**

Memory Block	Size	List of Jobs
Block 1	50k	**Job 3**
Block 2	200k	**Job 5**
Block 3	70k	**Job 7**
Block 4	115k	**Job 1**
Block 5	15k	**Job 6**

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	**Job 9**
Block 3	70k	**Job 8**
Block 4	115k	**Job 1**
Block 5	15k	

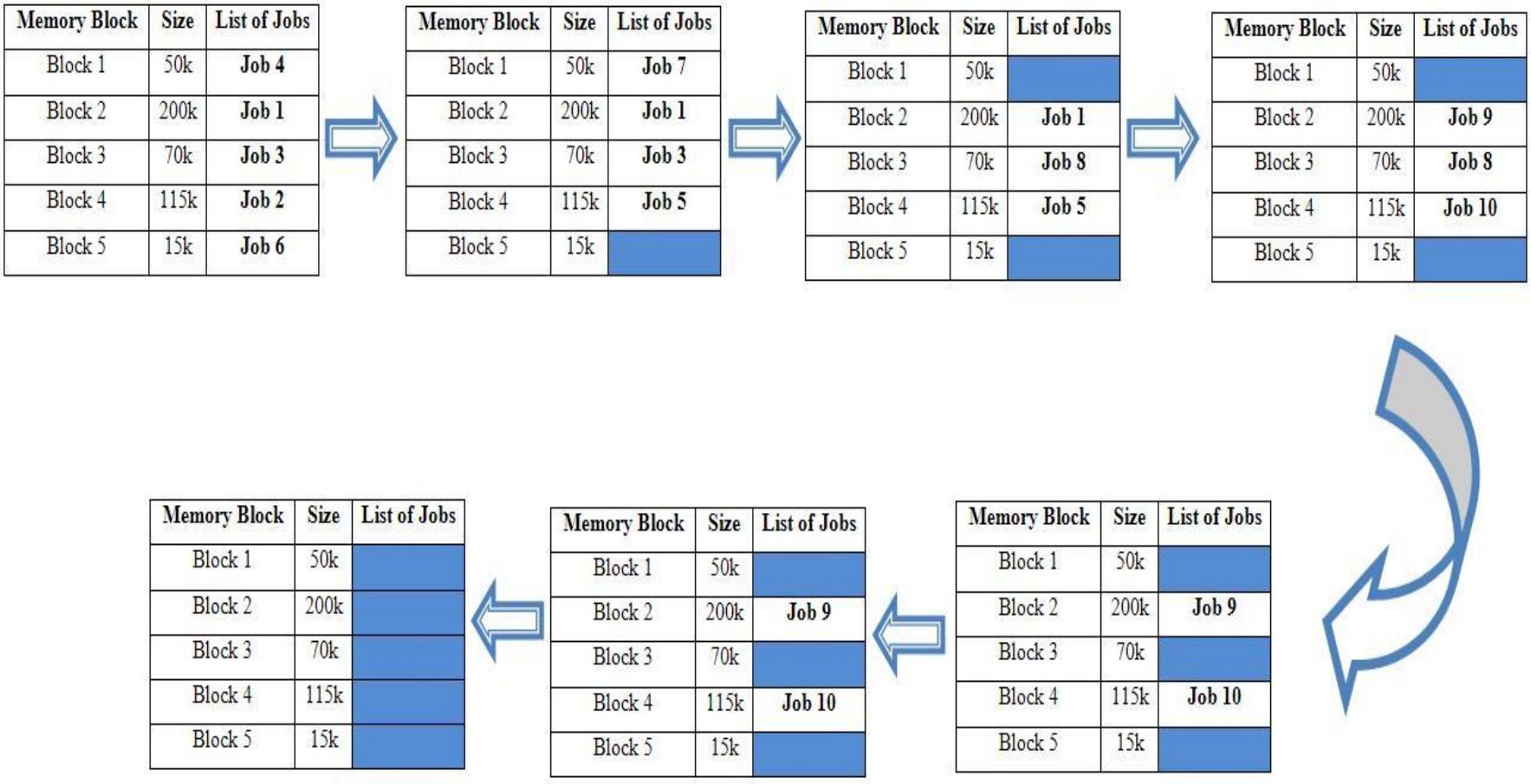
Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	**Job 9**
Block 3	70k	**Job 8**
Block 4	115k	**Job 10**
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	**Job 9**
Block 3	70k	
Block 4	115k	**Job 10**
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	
Block 3	70k	
Block 4	115k	**Job 10**
Block 5	15k	

Memory Block	Size	List of Jobs
Block 1	50k	
Block 2	200k	
Block 3	70k	
Block 4	115k	
Block 5	15k	

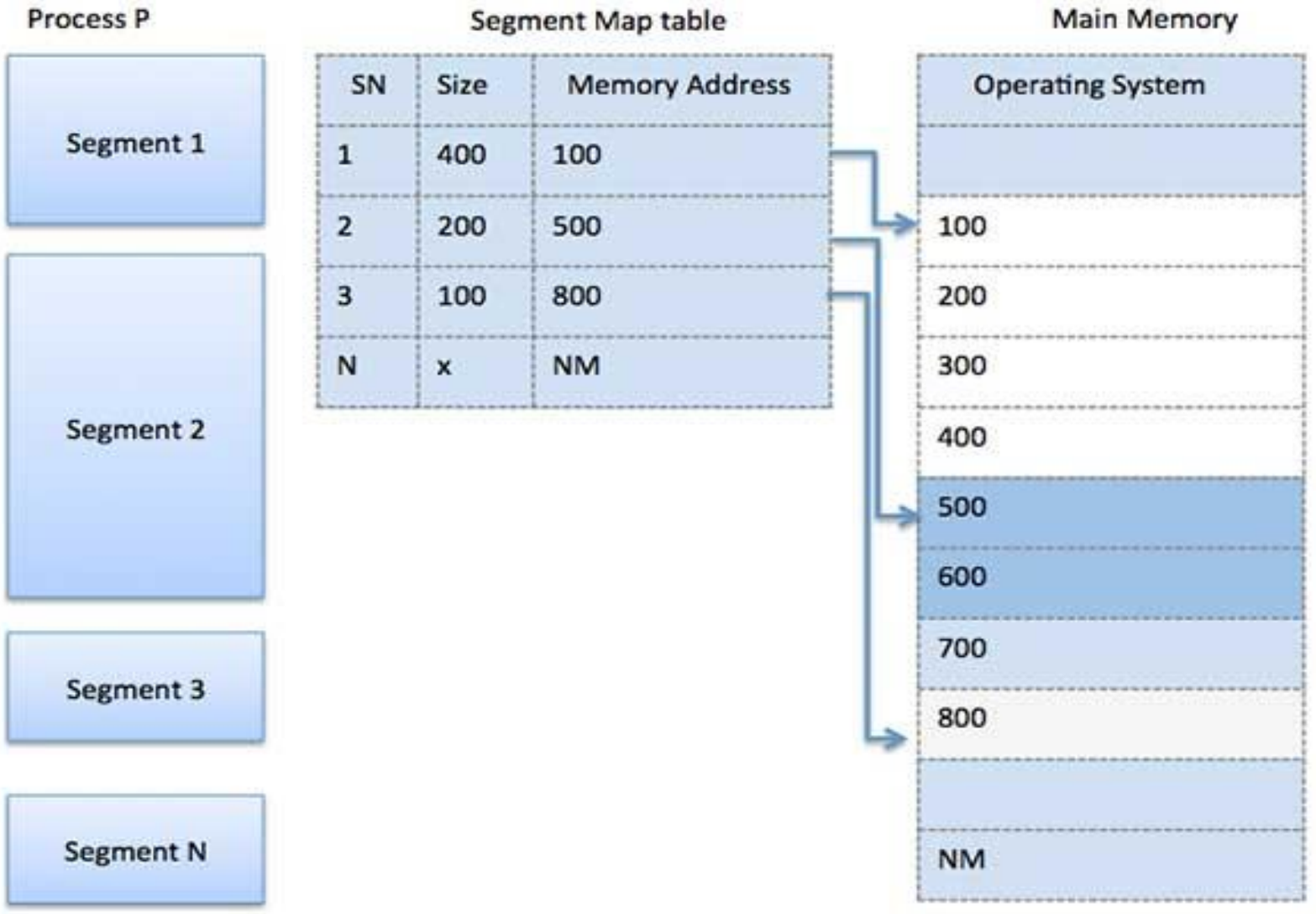
Example (Worst Fit)



Segmentation

- Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions.
- Each segment is actually a different logical address space of the program.
- When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory.
- A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a **segment map table** for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory.
- For each segment, the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.

Segmentation



THANK YOU